

# DOC

August 3, 2026

## 1 An Infinite Jazz Music Generator

**Originally posted:** 02-Sept-2018 **Last modified:** Dec 31, 2018 @ 10:21 am **Computer Music**  
| **Comments:** 1

**Update 23-Dec-2018:** If you want to see where this project went and the compositions that have resulted, check out the Algo-Jazz Series.

I recently became a bit more interested in how the bass behaves in various styles of music, and eventually ended up looking at walking bass lines. After messing around with my bass guitar for several hours, I wanted some algorithmic walking bass – and then I quickly wanted infinite walking algo-bass. Then I wanted algo-chords with it. So, I set about writing a relatively small Haskell program that:

1. Generates an infinite chord progression of Roman numerals,
2. Creates a walking bassline to that progression, and then
3. Adds simple chords over the top.

A small aside before going into the algorithm itself: no, this does **NOT** use Kulitta. If you are familiar with [Kulitta](#), however, you might notice a few types I’ve introduced in the code that seem rather similar to things in Kulitta, at least one of which (namely the type used for Roman numerals) isn’t actually critical to the algorithm itself and is, rather, a method to eventually hook this code up to other systems. The code here will likely be rolled into Kulitta as a foreground option in the future. In fact, many of Kulitta’s features were developed exactly like this – by playing with ideas in isolation of the larger system before being further refined, refactored, and eventually integrated.

So, how does it work? Given a series of Roman numerals, with one Roman numeral per measure, the walking bass algorithm is as follows:

1. Map each Roman numeral to a mode, and then find its root. Then, stochastically path find through the bass’s range to choose particular pitches for each root instance (remember, a root will be just something like C – it needs to go in an octave). For example, [II, V, I] in C-major has root pitch class numbers [2, 7, 0] and might become [38, 43, 48] :: [AbsPitch]. We will assume that each of these pitches happens at the start of a measure.
2. Given this backbone of bass pitches, we need to add three more bass notes per measure to achieve a walking bass. This uses two iterations of the same process. For each of those iterations, we look at the two adjacent, existing pitches,  $p_1$  and  $p_2$ . If  $p_1$  and  $p_2$  are too close together to add a unique pitch between them, then we choose a new pitch to insert between them that is above or below them. If  $p_1$  and  $p_2$  are exactly a major second apart, we add a chromatic tone between them. If  $p_1$  and  $p_2$  are farther apart, then we pick a pitch between

them – that with some probability is chromatic, but otherwise is a member of the current mode.

3. Once we have a list of pitches for the bass, one per beat, we stochastically add some ornaments.

The process for generating chords is much simpler: they are simply chosen by taking the pitch classes corresponding to the root, third, fifth, and seventh of the current mode and fitting them to the range 60–71, which is a deterministic process. Finally, some rhythm is stochastically added when the chords are converted to Euterpea’s `Music` values. Now, let’s get into the code!

First, we’ll start with the module statement and imports. This program requires the [Euterpea library](#) for representing score-level music in Haskell. Mac and Linux users: you will need a MIDI Synthesizer, like `SimpleSynth` or `FluidSynth` to hear the music – and it must be open and running before you start `GHC` or `GHCi`. Windows comes with a default synthesizer.

```
module Main where
```

```
import Euterpea
import System.Random
```

Jazz lead sheets are annotated with abstract chord symbols to indicate the harmony. We’ll keep it simple for now and just use the seven basic roman numerals, where there is one numeral for each scale index.

```
data RNum = I | II | III | IV | V | VI | VII
    deriving (Eq, Show, Ord, Enum)
```

Each Roman numeral also corresponds to a mode. For example, if we’re in C-major, then a I-chord is based on the C-major scale and a II-chord is based on Dorian rooted at D.

We’ll also define some type synonyms to help make the functions to come a bit more readable. We’ll use the type `PitchSpace` to indicate a collection of pitches. In blackboard math, this would be a set, but we’ll use a list. What exactly a pitch space represents is context dependent, so think of it as all the pitches we have to pick from at some point in time. Sometimes that might be all the pitches available to a particular instrument, but other times it might be only those pitches which satisfy a particular mode within that same range.

```
type PitchSpace = [AbsPitch]
```

Euterpea’s `AbsPitch` type is a synonym for `Int`. It is short for “Absolute Pitch” and is a music theoretic concept sometimes also called pitch numbers. In Euterpea, we can use `AbsPitch` values in the range 0–127.

We also need a way to represent the scale corresponding to the set of pitch classes associated with each mode. We’ll make a type called `ScalePCs` for this, which will also be a list of `AbsPitch` values corresponding to each degree in the scale starting with the root. However, these will be assumed to be more like pitch classes and therefore restricted to the range 0–11.

(A bit of music theory background: C4 is a pitch that would be `60 :: AbsPitch` in Euterpea. However, just C by itself is a pitch class – it doesn’t have an octave so we typically refer to it numerically as 0.)

```
type ScalePCs = [AbsPitch]
```

So, C-major will be:

```
[0,2,4,5,7,9,11] :: ScalePCs
```

D-major would be [2,4,6,7,9,11,1] and D-dorian, which is simply a rotation of C-major, would be [2,4,5,7,9,11,0].

Now, we can define a list of `ScalePCs` lists where each one corresponds to a Roman numeral, starting from C-major, or I. This is simply a matter of rotating the lowest pitch class number to the top for several iterations.

```
cModeScalePCs :: [ScalePCs]
cModeScalePCs = allRots [0,2,4,5,7,9,11]
  where
    allRots x = take 7 $ iterate doRot x
    doRot x   = map (`mod` 12) (tail x ++ [head x + 12])
```

Recall that we need to make some stochastic choices as part of the generation. A lot of those in the code to follow involve choosing something from a list. So, we'll write a function called `choose` to accomplish that task.

```
choose :: StdGen -> [a] -> (StdGen, a)
choose g xs =
  let (r, g') = next g
  in (g', xs !! (r `mod` length xs))
```

To avoid jumping right into the IO monad, we need to thread a random generator of type `StdGen` throughout many of the type signatures. We will do this in two ways: for functions like the one above that have a finite number of possible steps, we will return a new generator as part of the result. However, functions that need to address infinite outputs must be written in a tail-recursive way that requires “consuming” the input generator and not returning a new one with the output.

We will start with a function to do a simple task: given a pitch space for the bass (all the pitches it can play), the last pitch played, and the next target pitch class, we will randomly select a pitch of that pitch class within an octave of the last pitch (assuming one exists). Because only one choice is made here, we can return a new generator along with the chosen pitch.

```
nextBassRoot :: PitchSpace -> AbsPitch -> AbsPitch -> StdGen -> (StdGen, AbsPitch)
nextBassRoot pSpace lastPitch nextPC g =
  let nextPitches = filter (\x -> x `mod` 12 == nextPC) pSpace
      nextPitchesX = filter (\x -> abs(x - lastPitch) <= 12) nextPitches
  in if null nextPitchesX
     then choose g nextPitches
     else choose g nextPitchesX
```

Now, if we want to do this for an infinite series of pitch classes, we have to use the tail-recursive style and “consume” the input generator. Supposing we have a list of pitch classes representing the roots of each mode in our infinite chord progression, we turn them into concrete pitches as follows.

```
genBassRoots :: PitchSpace -> AbsPitch -> StdGen -> [AbsPitch] -> [AbsPitch]
genBassRoots pSpace lastPitch g [] = []
genBassRoots pSpace lastPitch g (x:xs) =
  let (g1, x1) = nextBassRoot pSpace lastPitch x g
  in x1 : genBassRoots pSpace x1 g1 xs
```

Now we address the iterative process of filling in pitches between occurrences of the roots. Recall from the algorithm description that we have three ways to add a new pitch between two existing pitches:

1. Add a chromatic step between two pitches (if they are distance 2)
2. Choose a nearby note outside the two pitches
3. Choose a pitch between the two pitches if they are far enough apart

The first one is easy since it is deterministic, but the others involve stochastic decisions. So, we introduce two functions: one for choosing a note within a fifth (7 half steps) on either side of the two pitches and another for selecting a pitch in the middle.

```
chooseNearby :: PitchSpace -> AbsPitch -> AbsPitch -> StdGen -> (StdGen, AbsPitch)
chooseNearby pSpace p1 p2 g =
  let pSpaceA = filter (\x -> x > p1 - 7 && x < p2 + 7 && x /= p1 && x /= p2) pSpace
      pSpaceB = filter (\x -> x /= p1 && x /= p2) [p1 - 7 .. p2 + 7]
  in choose g (if null pSpaceA then pSpaceB else pSpaceA)
```

```
chooseBetween :: PitchSpace -> AbsPitch -> AbsPitch -> StdGen -> (StdGen, AbsPitch)
chooseBetween pSpace p1 p2 g =
  let pSpaceA = filter (\x -> x > p1 && x < p2) pSpace
      pSpaceB = if null pSpaceA then [p1..p2] else pSpaceA
  in choose g pSpaceB
```

Now we employ these functions in a function to fill in one pitch between two existing pitches. We will allow some probability of the new pitch being chosen from a chromatic scale to add some dissonance. Otherwise, it will be strictly selected from the current mode of the first pitch. The probability of allowing chromaticism is left as its own constant so that it can be easily changed for experimentation. Try varying it and observe how it changes the flavor of the bassline.

```
probChromatic :: Double
probChromatic = 0.2
```

```
fill1 :: PitchSpace -> ScalePCs -> AbsPitch -> AbsPitch -> StdGen -> (StdGen, AbsPitch)
fill1 pSpace mode p1 p2 g =
  let pSpaceM = filter (\p -> elem (mod p 12) mode) pSpace
      (r, g1) = randomR (0.0, 1.0 :: Double) g
      thePSpace = if r < probChromatic then pSpace else pSpaceM
  in if abs(p2 - p1) == 2
      then (g, min p1 p2 + 1) -- distance 2? Add a chromatic step
      else if abs(p2 - p1) <= 1
          then chooseNearby thePSpace (min p1 p2) (max p1 p2) g -- no room? Choose nearby
          else chooseBetween thePSpace (min p1 p2) (max p1 p2) g -- otherwise, choose a mi
```

We need to perform the function above for each adjacent pair of pitches in a list – and then do the exact same thing again. The first pass will fill in pitches that will eventually fall on beat 3, and the second pass will fill in those on beats 2 and 4.

Instead of taking a list of just the pitches to operate on, since decisions are made based on the current mode, we will take a list of pitches paired with their current modes.

```
fillRec :: PitchSpace -> [(AbsPitch, ScalePCs)] -> StdGen -> (StdGen, [(AbsPitch, ScalePCs)])
```

```

fillRec pSpace ((x1, mode1):(x2, mode2):xs) g =
  let (g1, newP)      = fill1 pSpace mode1 x1 x2 g
      (g2, otherPs) = fillRec pSpace ((x2, mode2):xs) g1 -- recursive call
  in (g2, (x1, mode1) : (newP, mode1) : otherPs)
fillRec pSpace x g = (g, x)

```

Now, at this point we have to be careful. We can't actually call the function above on a raw infinite list and still be able to play the result in real time with lazy evaluation – it would hang even if we re-wrote it to be tail-recursive. However, we have a workaround, which is to apply it over chunks of the sequence. Because decisions in one measure do **NOT** affect decisions made in the next measure with this algorithm, we can safely handle an infinite list by working on it one measure at a time.

Given two pitches, one at the start of measure  $i$  and the other at the start of measure  $i + 1$ , we fill in the bassline for measure  $i$  with the mode for measure  $i$ .

```

nextBassMeasure :: PitchSpace -> ScalePCs -> AbsPitch -> AbsPitch -> StdGen -> (StdGen, ScalePCs)
nextBassMeasure pSpace mode p1 p2 g =
  let (g1, bassLayer1) = fillRec pSpace [(p1, mode), (p2, mode)] g
      (g2, bassLayer2) = fillRec pSpace bassLayer1 g1
  in (g2, take (length bassLayer2 - 1) $ map fst bassLayer2)

```

This function can be called recursively over a potentially infinite list. We again take a list of pitch and mode pairs, where each pitch is an instance of the mode's root.

```

bassGenRec :: PitchSpace -> [(AbsPitch, ScalePCs)] -> StdGen -> [AbsPitch]
bassGenRec pSpace ((x1, mode1):(x2, mode2):xs) g =
  let (g1, nextPs) = nextBassMeasure pSpace mode1 x1 x2 g
  in nextPs ++ bassGenRec pSpace ((x2, mode2):xs) g1
bassGenRec pSpace x g = map fst x

```

Finally, we can convert the bassline to one of Euterpea's Music types. We'll use `Music (AbsPitch, Volume)`, which was introduced as a playable type in Euterpea 2.0.5. There is a 70% chance of leaving each note unaltered, and then a 10% chance of each of the following being added as a small, grace-note like addition leading into the next pitch:

1. Approaching the next pitch chromatically from above
2. Approaching the next pitch chromatically from below
3. Repeating the next pitch.

```

bassToMusic :: [AbsPitch] -> StdGen -> Music (AbsPitch, Volume)
bassToMusic [] g = rest 0
bassToMusic [x] g = note wn (x, 120)
bassToMusic (x1:x2:xs) g =
  let (r, g1) = randomR (0.0, 1.0 :: Double) g
      m = if r < 0.7
          then note qn (x1, 120)
          else if r < 0.8
              then note (3*sn) (x1, 120) :+: note (sn) (x2 + 1, 80)
              else if r < 0.9
                  then note (3*sn) (x1, 120) :+: note (sn) (x1, 80)
                  else note (3*sn) (x1, 120) :+: note (sn) (x2 - 1, 80)
  in m :+: bassToMusic (x2:xs) g1

```

Now, we also need to convert chords to music. You'll notice this is the first time chords appear. That's because they're really easy to generate and have little in the way of stochastic decisions. In fact, the only stochastic choice is the rhythm assigned to them in a `Music` value. Otherwise, they are deterministically taken directly from the same list of `ScalePCs` that is also used to interpret the bass in every measure.

```
chordsToMusic :: [[AbsPitch]] -> StdGen -> Music (AbsPitch, Volume)
chordsToMusic [] g = rest 0
chordsToMusic [x] g = chord $ map (\p -> note wn (p, 80)) x
chordsToMusic (x:xs) g =
  let (g1, durs) = choose g [[hn, hn], [2*qn/3, wn - 2*qn/3], [dhn, qn]]
      fx d       = chord $ map (\p -> note d (p, 80)) x
      m         = line $ map fx durs
  in m :+: chordsToMusic xs g1
```

Now we're ready to put all these things together using the original algorithm described at the top. The arguments to this function are: the root of the desired home key (i.e. 7 for G major or minor), whether the home key is major, a potentially infinite list of roman numerals, and a random generator.

*Note: because in this code we're taking a truly random sequence of Roman numerals, Major and Minor won't sound tremendously different. However, if you supply your own patterns that more heavily use common progressions like ii-V-I, it will sound distinctly different.*

```
makeJazz :: AbsPitch -> Bool -> [RNum] -> StdGen -> Music (AbsPitch, Volume)
makeJazz root isMajor rNums g =
  let modelList = if isMajor
                  then cModeScalePCs -- using major
                  else drop 5 cModeScalePCs ++ take 5 cModeScalePCs -- handle minor selection
      bassRange   = [30..50]
      initScalePCss = map ((cModeScalePCs !!) . fromEnum) rNums -- turn numerals into C-major
      ms          = map (map ((`mod` 12) . (+root))) $ initScalePCss -- transpose modes b
      ms0         = map head ms -- get mode roots
      gs          = infSplit g -- set up an infinite supply of generators
      (gsp, startPoint) = choose (gs !! 0) bassRange -- pick a general starting point for the b
      initBassPitches0 = genBassRoots bassRange startPoint gsp ms0 -- choose measure nodes
      bassMeasurePs    = bassGenRec bassRange (zip initBassPitches0 ms) (gs !! 1) -- gen bass f
      bassMusic        = bassToMusic bassMeasurePs (gs !! 2) -- convert bass to Music with orn
      chords           = map (map (+60)) $ -- transposition above bassline
                        map (\x -> [x !! 0, x !! 2, x !! 4, x !! 6]) ms -- form a simple seven
      chordMusic       = chordsToMusic chords (gs !! 3) -- convert chords to Music with rhythm
  in instrument AcousticBass bassMusic :=: instrument RhodesPiano chordMusic
where
  infSplit :: StdGen -> [StdGen] -- necessary to get many generators from just one
  infSplit g = let (g1, g2) = split g in g1 : infSplit g2
```

And finally, you can run the main function below in GHCi to hear as much silly algo-jazz as you could possibly want. Use **Ctrl+C** to exit out of the infinite playback (note: you may have to do that a couple times in rapid succession on some machines to make it stop).

```
majorHomeKey :: Bool
```

```

majorHomeKey = True -- change to false to get a minor home key

homeKeyRoot :: AbsPitch
homeKeyRoot = 0 -- change to another pitch class number if desired (ex: G=7)

main :: IO ()
main = do
  gR <- newStdGen
  gM <- newStdGen
  let allRNums = enumFrom I
      chordProg = map (allRNums !!) $ randomRs (0, length allRNums - 1) gR
      m         = makeJazz homeKeyRoot majorHomeKey chordProg gM
  play m

```

Should you want to do something finite or to manually control the Roman numeral list in some way, you can do something like this instead. In fact, the code snippet below is how I generated the segment in the video at the top of the page:

```

allRNums = enumFrom I
infProg  = map (allRNums !!) $ randomRs (0, length allRNums - 1) (mkStdGen 123)
shortProg = take 8 infProg ++ [I]
shortJazz = makeJazz homeKeyRoot majorHomeKey shortProg (mkStdGen 456)

```

And then use this in GHCi to write a MIDI file:

```
writeMidi "test.mid" shortJazz
```

For those interested in playing with this in their own text editors, here's all the code from above in one big, horrible, un-commented lump. It's in LHS format with the ">" in front of each line of code, because that seems to be the only reliable way to preserve the formatting across post edits in wordpress. Create a .lhs file, copy/paste the code below into it, save, and load it in GHCi. The total line count from the first ">" is 131. In case anyone saw my Facebook post about having a simple jazz generator in <100 lines of code before viewing this page, the older, smaller version did not support infinitely long music and also lacked the stochastic chromaticism that gives nice flavor to the longer version's output. The extra lines were worthwhile additions.

|                                       |
|---------------------------------------|
| Infinite Jazz Generator – Donya Quick |
|---------------------------------------|

```

> module Main where

> import Euterpea
> import System.Random

> data RNum = I | II | III | IV | V | VI | VII
>   deriving (Eq, Show, Ord, Enum)

> type PitchSpace = [AbsPitch]
> type ScalePCs = [AbsPitch]

> cModeScalePCs :: [ScalePCs]

```



```

> cModeScalePCs = allRots [0,2,4,5,7,9,11]
> where
>   allRots x = take 7 $ iterate doRot x
>   doRot x   = map (`mod` 12) (tail x ++ [head x + 12])

> choose :: StdGen -> [a] -> (StdGen, a)
> choose g xs =
>   let (r, g') = next g
>   in (g', xs !! (r `mod` length xs))

> nextBassRoot :: PitchSpace -> AbsPitch -> AbsPitch -> StdGen -> (StdGen, AbsPitch)
> nextBassRoot pSpace lastPitch nextPC g =
>   let nextPitches = filter (\x -> x `mod` 12 == nextPC) pSpace
>       nextPitchesX = filter (\x -> abs(x - lastPitch) <= 12) nextPitches
>   in if null nextPitchesX
>       then choose g nextPitches
>       else choose g nextPitchesX

> genBassRoots :: PitchSpace -> AbsPitch -> StdGen -> [AbsPitch] -> [AbsPitch]
> genBassRoots pSpace lastPitch g [] = []
> genBassRoots pSpace lastPitch g (x:xs) =
>   let (g1, x1) = nextBassRoot pSpace lastPitch x g
>   in x1 : genBassRoots pSpace x1 g1 xs

> chooseNearby :: PitchSpace -> AbsPitch -> AbsPitch -> StdGen -> (StdGen, AbsPitch)
> chooseNearby pSpace p1 p2 g =
>   let pSpaceA = filter (\x -> x > p1 - 7 && x < p2 + 7 && x /= p1 && x /= p2) pSpace
>       pSpaceB = filter (\x -> x /= p1 && x /= p2) [p1 - 7 .. p2 + 7]
>   in choose g (if null pSpaceA then pSpaceB else pSpaceA)

> chooseBetween :: PitchSpace -> AbsPitch -> AbsPitch -> StdGen -> (StdGen, AbsPitch)
> chooseBetween pSpace p1 p2 g =
>   let pSpaceA = filter (\x -> x > p1 && x < p2) pSpace
>       pSpaceB = if null pSpaceA then [p1..p2] else pSpaceA
>   in choose g pSpaceB

> probChromatic :: Double
> probChromatic = 0.2

> fill1 :: PitchSpace -> ScalePCs -> AbsPitch -> AbsPitch -> StdGen -> (StdGen, AbsPitch)
> fill1 pSpace mode p1 p2 g =
>   let pSpaceM = filter (\p -> elem (mod p 12) mode) pSpace
>       (r, g1) = randomR (0.0, 1.0 :: Double) g
>       thePSpace = if r < probChromatic then pSpace else pSpaceM
>   in if abs(p2 - p1) == 2
>       then (g, min p1 p2 + 1) -- distance 2? Add a chromatic step
>       else if abs(p2 - p1) <= 1
>           then chooseNearby thePSpace (min p1 p2) (max p1 p2) g -- no room? Choose nearby

```



```

>         else chooseBetween thePSpace (min p1 p2) (max p1 p2) g -- otherwise, choose a
>
> fillRec :: PitchSpace -> [(AbsPitch, ScalePCs)] -> StdGen -> (StdGen, [(AbsPitch, ScalePCs)])
> fillRec pSpace ((x1, mode1):(x2, mode2):xs) g =
>   let (g1, newP) = fill1 pSpace mode1 x1 x2 g
>       (g2, otherPs) = fillRec pSpace ((x2, mode2):xs) g1 -- recursive call
>   in (g2, (x1, mode1) : (newP, mode1) : otherPs)
> fillRec pSpace x g = (g, x)
>
> nextBassMeasure :: PitchSpace -> ScalePCs -> AbsPitch -> AbsPitch -> StdGen -> (StdGen, ScalePCs)
> nextBassMeasure pSpace mode p1 p2 g =
>   let (g1, bassLayer1) = fillRec pSpace [(p1, mode), (p2, mode)] g
>       (g2, bassLayer2) = fillRec pSpace bassLayer1 g1
>   in (g2, take (length bassLayer2 - 1) $ map fst bassLayer2)
>
> bassGenRec :: PitchSpace -> [(AbsPitch, ScalePCs)] -> StdGen -> [AbsPitch]
> bassGenRec pSpace ((x1, mode1):(x2, mode2):xs) g =
>   let (g1, nextPs) = nextBassMeasure pSpace mode1 x1 x2 g
>   in nextPs ++ bassGenRec pSpace ((x2, mode2):xs) g1
> bassGenRec pSpace x g = map fst x
>
> bassToMusic :: [AbsPitch] -> StdGen -> Music (AbsPitch, Volume)
> bassToMusic [] g = rest 0
> bassToMusic [x] g = note wn (x, 120)
> bassToMusic (x1:x2:xs) g =
>   let (r, g1) = randomR (0.0, 1.0 :: Double) g
>       m = if r < 0.7
>           then note qn (x1, 120)
>           else if r < 0.8
>               then note (3*sn) (x1, 120) :+: note (sn) (x2 + 1, 80)
>               else if r < 0.9
>                   then note (3*sn) (x1, 120) :+: note (sn) (x1, 80)
>                   else note (3*sn) (x1, 120) :+: note (sn) (x2 - 1, 80)
>   in m :+: bassToMusic (x2:xs) g1
>
> chordsToMusic :: [[AbsPitch]] -> StdGen -> Music (AbsPitch, Volume)
> chordsToMusic [] g = rest 0
> chordsToMusic [x] g = chord $ map (\p -> note wn (p, 80)) x
> chordsToMusic (x:xs) g =
>   let (g1, durs) = choose g [[hn, hn], [2*qn/3, wn - 2*qn/3], [dhn, qn]]
>       fx d = chord $ map (\p -> note d (p, 80)) x
>       m = line $ map fx durs
>   in m :+: chordsToMusic xs g1
>
> makeJazz :: AbsPitch -> Bool -> [RNum] -> StdGen -> Music (AbsPitch, Volume)
> makeJazz root isMajor rNums g =
>   let modeList = if isMajor
>   then cModeScalePCs -- using major

```

```

>         else drop 5 cModeScalePCs ++ take 5 cModeScalePCs -- handle minor selecti
>     bassRange      = [30..50]
>     initScalePCss  = map ((cModeScalePCs !!) . fromEnum) rNums -- turn numerals into C-m
>     ms             = map (map (`mod` 12) . (+root))) $ initScalePCss -- transpose modes
>     ms0            = map head ms -- get mode roots
>     gs             = infSplit g -- set up an infinite supply of generators
>     (gsp, startPoint) = choose (gs !! 0) bassRange -- pick a general starting point for the
>     initBassPitches0 = genBassRoots bassRange startPoint gsp ms0 -- choose measure nodes
>     bassMeasurePs   = bassGenRec bassRange (zip initBassPitches0 ms) (gs !! 1) -- gen bas
>     bassMusic       = bassToMusic bassMeasurePs (gs !! 2) -- convert bass to Music with o
>     chords          = map (map (+60)) $ -- transposition above bassline
>                     map (\x -> [x !! 0, x !! 2, x !! 4, x !! 6]) ms -- form a simple se
>     chordMusic      = chordsToMusic chords (gs !! 3) -- convert chords to Music with rhyt
> in instrument AcousticBass bassMusic :=: instrument RhodesPiano chordMusic
> where
>     infSplit :: StdGen -> [StdGen] -- necessary to get many generators from just one
>     infSplit g = let (g1, g2) = split g in g1 : infSplit g2

> majorHomeKey :: Bool
> majorHomeKey = False -- change to false to get a minor home key

> homeKeyRoot :: AbsPitch
> homeKeyRoot = 0 -- change to another pitch class number if desired (ex: G=7)

> main :: IO ()
> main = do
>     gR <- newStdGen
>     gM <- newStdGen
>     let allRNums = enumFrom I
>         chordProg = map (allRNums !!) $ randomRs (0, length allRNums - 1) gR
>         m         = makeJazz homeKeyRoot majorHomeKey chordProg gM
>     play m

```

[ ]: